COMP 3920 Database Systems 1

Sequelize ORM

Introduction:

The goal of this document is to serve as a walkthrough of working with an Object Relational Mapping (ORM) called Sequelize. Sequelize is a data abstraction layer that creates objects in Node.js which you can use to retrieve and modify data in a database. In Sequelize, you create a model for each table in your database that you want to interact with. When you want to add or edit a row inside one of those tables, you instantiate an object based on that model you created. With these models and the objects created from them, Sequelize handles all the SQL command required to SELECT, INSERT, UPDATE and DELETE from the tables!

The beauty of using Sequelize, is this **same code** can be used across multiple different RDBMS systems including: MS SQL, MySQL, MariaDB, PostgreSQL and SQLite.

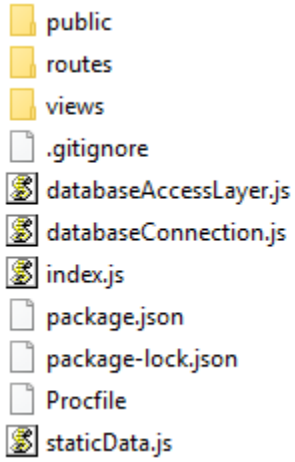
Also, because we have been using an EJS template, we don't need to be changing our HTML views at all either.

The provided starting code is able to SELECT from a list of web users as well as add new and remove users. We will convert it from using direct SQL commands for retrieving and changing the data (SELECT, INSERT, UPDATE and DELETE) to

Yet another great reason to use Sequelize is that it can handle preventing SQL injection attacks. This is important because we still want to maintain a high level of security, like we learned from previous labs.

Step 1: Download the `project files.zip` zip file from D2L (learn.bcit.ca).

Extract all files from the zip file into a folder. Your folder should look like this:



Step 2: Create a new GitHub repo and host your site.

Use a service like Render to host your site.

Link your hosted site to your new GitHub repo so that it can automatically deploy when new code is pushed to the repo.

Edit the `sample.env` file with the correct User, Password, Host and Database and rename it to `.env`. Make sure your `.env` file it added to your `.gitignore` file so that it never makes it into your git repo. You can use details for your localhost or a Remote Hosted DB database (like Aiven.io) when testing locally. When deploying to your hosted site, though, you'll need to provide the hosting site the environment variables for your Remote Hosted DB details.

Create a new git commit with the above changes for the correct database connection details and push them to GitHub.

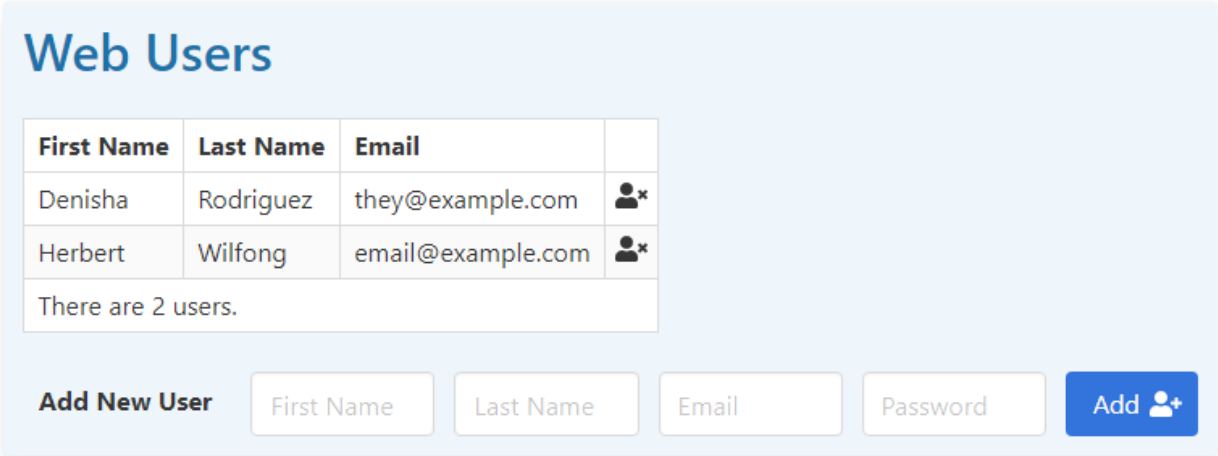
IMPORTANT! Remember - if you don't commit your code to git **AND Push your code to GitHub**, your hosted site won't have access to the latest code.

Step 3: Import the required database, tables and data.

Download the `pets.sql` file from the Learning Hub and import it into your Remote Hosted DB database (and your localhost database if you want to test it locally).

Step 4: Test your hosted site.

The code provided used a `web_user` table to display to the screen.



For the first part of this we won't be adding any new functionality, just replacing the database model code with Sequelize.

Step 5: Install Sequelize for Node.

On a command line, run:
`npm install sequelize`

```
C:\Users\User\Documents\node code\comp2350-week10-test1>npm install sequelize
npm WARN tutorial@1.0.0 No description
npm WARN tutorial@1.0.0 No repository field.
npm WARN optional SKIPPING OPTIONAL DEPENDENCY: fsevents@2.3.1 (node_modules\fseve
nts):
npm WARN notsup SKIPPING OPTIONAL DEPENDENCY: Unsupported platform for fsevents@2.
3.1: wanted {"os":"darwin","arch":"any"} (current: {"os":"win32","arch":"x64"})

+ sequelize@6.5.1
added 17 packages from 79 contributors and audited 223 packages in 20.965s

11 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Step 6: Create a connection string for Sequelize.

Create a file called `databaseConnectionSequelize.js` with the following contents:

```
require('dotenv').config();

const dbConfigLocal =
`mysql://${process.env.DB_USER}:${process.env.DB_PASSWORD}@${process.en
v.DB_HOST}/${process.env.DB_NAME}`;

var databaseConnectionString = dbConfigLocal;

module.exports = databaseConnectionString;
```

After the variable replacement, this will give us a connection string that will look like something like this:
`mysql://root:Password@localhost/lab_example`

For your hosted database, you can build a connection string from your Remote Hosted DB database connection details - something like:
`mysql://freedb_2350_main:w1c4poka4*x4w32t@ sql.freedb.tech:3306/ freedb_comp2350-A87654321`

Step 7: Create a Sequelize model for the `web_user` table.

Create a folder that will contain the models for this example called `models`.

Create the following file with the following contents:

File - `models\web_user.js`:

```
const {Sequelize, DataTypes} = require('sequelize');
const databaseConnectionString = include('/databaseConnectionSequelize');
const sequelize = new Sequelize(databaseConnectionString);

const userModel = sequelize.define('web_user',
       {web_user_id: {type: Sequelize.INTEGER, autoIncrement: true,
primaryKey: true},
        first_name: {type: Sequelize.STRING, allowNull: false},
        last_name: {type: Sequelize.STRING, allowNull: false},
        email: {type: Sequelize.STRING, allowNull: false},
        password_hash: {type: Sequelize.STRING, allowNull: true}
       },
       {tableName: 'web_user',
        timestamps: false,
        singular: 'web_user',
        plural: 'web_user'}
);

module.exports = userModel;
```

Before we move on here, it's important to look at the contents of this model.

Let's examine each line to get a sense of what is happening on it.

Line 1: `const {Sequelize, DataTypes} = require('sequelize');`

This line just imports the Sequelize library and let's us use the Sequelize object which handles the connection to the database and the DataTypes object which defines some constants for handling the different database data types like STRING which will get translated to `VARCHAR()` in MySQL.

Line 2: `const databaseConnectionString = include('/databaseConnectionSequelize');`

This line gets the connection string to the database (remember this will be either the hosted connection string or the local one).

Line 3: `const sequelize = new Sequelize(databaseConnectionString);`

This line initializes Sequelize to use the correct connection string from the previous step.

Line 4: `const userModel = sequelize.define('web_user',`

This line creates the model for the `web_user` table.

Lines 5-10:
```
{web_user_id: {type: Sequelize.INTEGER, autoIncrement: true, primaryKey:
    true},
 first_name: {type: Sequelize.STRING, allowNull: false},
 last_name: {type: Sequelize.STRING, allowNull: false},
 email: {type: Sequelize.STRING, allowNull: false},
 password_hash: {type: Sequelize.STRING, allowNull: true},
 password_salt: {type: Sequelize.STRING, allowNull: false}
},
```

These lines creates a model for the table and all its columns. Here we have:

Column Name	Data Type	Allows Null	Extras
web_user_id	Integer	No	auto_increment, primary key
first_name	String	No	
last_name	String	No	
email	String	No	
password_hash	String	Yes	
password_salt	String	No	

Lines 11-14:
```
{tableName: 'web_user',
 timestamps: false,
 singular: 'web_user',
 plural: 'web_user'}
```

These lines are some options given to Sequelize for the model. It specifies the table name as 'web_user'. Also, well specify that sequelize shouldn't add a timestamp column to our table. Lastly, we'll provide Sequelize the singular and plural forms of the table name as `web_user`.
(Note: Sequelize has some weird defaults if we don't specify these options).

Line 16: `module.exports = userModel;`

This line exports our model so that it can be used in other modules as needed.

Step 8: Use the Sequelize Web User Model.

In order to use our Sequelize model instead of our existing database layer, we will change where we get our data from in the `routes/router.js` file.

First start by commenting out our existing database layer by commenting the following lines into `routes/router.js`:

```
const database = include('databaseConnection');
const dbModel = include('databaseAccessLayer');
```

Next, add the import for our model.

Add the following line:
`const userModel = include('models/web_user');`

The first few lines of the file should look like this:

```
routes > JS router.js > ...
1  const router = require('express').Router();
2  // const database = include('databaseConnection');
3  // const dbModel = include('databaseAccessLayer');
4  //const dbModel = include('staticData');
5  const userModel = include('models/web_user');
```

Comment out the 3 routes that we currently have for:

1. `router.get('/', (req, res) => {…});`
2. `router.post('/addUser', (req, res) => {…});`
3. `router.get('/deleteUser', (req, res) => {…});`

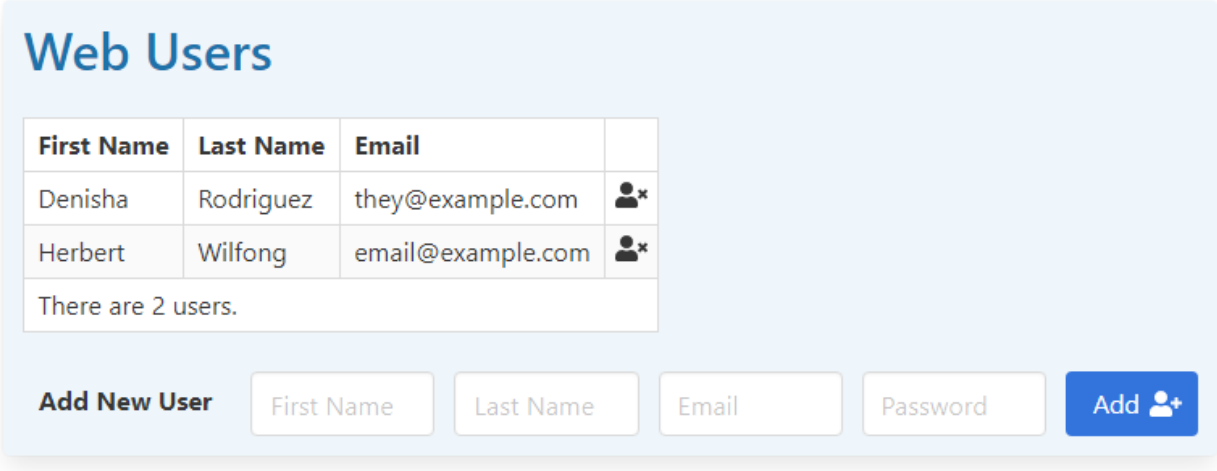
We will be recreating these routes, but this time, we'll use the Sequelize data layer instead of the direct SQL layer we created in previous labs.

So, for the main route at the root ('/') we need a to get all the users. Getting all the users from the userModel is quite simple, we just need to call `findAll()`. Additionally we can specify which columns we want by using the attributes variable inside the options object passed in. This way we won't return the columns we <u>don't need</u>, like `password_hash`.

In `routes/router.js` add a function to handle the `get` request for `/`.

```
router.get('/', async (req, res) => {
	console.log("page hit");
	try {
		const users = await userModel.findAll({attributes:
['web_user_id','first_name','last_name','email']}); //{where: {web_user_id:
1}}
		if (users === null) {
			res.render('error', {message: 'Error connecting to
MySQL'});
			console.log("Error connecting to userModel");
		}
		else {
			console.log(users);
			res.render('index', {allUsers: users});
		}
	}
	catch(ex) {
		res.render('error', {message: 'Error connecting to MySQL'});
		console.log("Error connecting to MySQL");
		console.log(ex);
	}
});
```

With these changes, your website should work (at least for showing all the users). Let's test it out!



Our Insert and Delete routes are broken right now, so lets fix them.

For the Delete route, we need to find the row in the database that corresponds to the id passed in the URL. We use the `findByPk(..)` function to on the `userModel` to get an object that points to that row. Once we have access to the object we can delete it using the `destroy()` method.

It's important to note that since we aren't dealing with SQL commands directly, Sequelize will handle all the SQL Injection Protection - meaning we don't need to worry about SQL Injection attacks!

In `routes/router.js` add a function to handle the `get` request for `/deleteUser`.

```
router.get('/deleteUser', async (req, res) => {
	try {
		console.log("delete user");

		let userId = req.query.id;
		if (userId) {
			console.log("userId: "+userId);
			let deleteUser = await userModel.findByPk(userId);
			console.log("deleteUser: ");
			console.log(deleteUser);
			if (deleteUser !== null) {
				await deleteUser.destroy();
			}
		}
		res.redirect("/");
	}
	catch(ex) {
		res.render('error', {message: 'Error connecting to MySQL'});
		console.log("Error connecting to MySQL");
		console.log(ex);
	}
});
```

Now we'll add the route to allow for inserting a new row.

We'll use BCrypt to hash the password so we need to require this module at the top of our router file.

Important note: `userModel.build(…)` just creates an in memory copy of the data we want to save to the database. It's not until we call the `.save()` method that the actual INSERT statement gets executed to save it to the database. If you wanted to do this in one single step, you could use the `userModel.create(…)` method (instead of using both the `.build(…)` and `.save()` methods).

Modify `routes/router.js` to add a few require statements (put these at the top):

```
const bcrypt = require('bcrypt');
```

Also in `routes/router.js` create a route to receive the `POST` request on `/addUser`:

```
router.post('/addUser', async (req, res) => {
	try {
		console.log("form submit");

		const password_hash = await bcrypt.hash(req.body.password, 12);

		let newUser = userModel.build(
			{
				first_name: req.body.first_name,
				last_name: req.body.last_name,
				email: req.body.email,
				password_salt: password_hash
			}
		);
		await newUser.save();
		res.redirect("/");
	}
	catch(ex) {
		res.render('error', {message: 'Error connecting to MySQL'});
		console.log("Error connecting to MySQL");
		console.log(ex);
	}
});
```

Awesome! So at this point we have completely replaced our Data Access Layer from using SQL directly to using Sequelize which takes care of all the SQL commands behind the scenes.

Let's also create the `pet` and `pet_type` tables so that our `web_users` can have some furry friends.

Step 9: Create the `pet` and `pet_type` tables.
pet:

Column Name	Data Type	Not Null	auto_increment	Key
pet_id	int	Yes	Yes	Primary
web_user_id	int	Yes		Foreign Key to web_user
name	varchar(50)	Yes		
pet_type_id	int	Yes		Foreign Key to pet_type

pet_type:

Column Name	Data Type	Not Null	auto_increment	Key
pet_type_id	int	Yes	Yes	Primary
type	varchar(50)	Yes		**Unique

****Enforce Unique** on the type column in the `pet_type` table so, for example, "dog" doesn't accidentally get added twice.

Add a few pets and some pet types for some sample data.

Step 10: Create a model for the `pet` and `pet_type` tables.

Similar to how we created our `userModel` in `models/web_user.js`, create a `models/pet.js` and `models/pet_type.js` files with the required fields to make the model work.

Step 11: Show all pets.

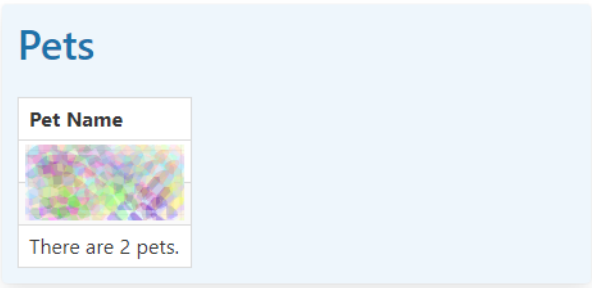
Create a really simple page called `pets.ejs` that is similar to `index.ejs`, but instead of displaying the `web_users`, display the name of every pet in the `pet` table. Note: Just display the pet name, don't worry about any other columns like the owner (`web_user_id`) or pet type (`pet_type_id`) just yet. Also, we don't need the form that allows for inserting new rows - you can delete the `<form></form>`.

Create a route for the `GET` request on `/pets` that will render the `pets.ejs` page.

Submit a screenshot of your Hosted App
with some users added to the database.

Submit

Filename: **01_Sequelize_Pets_Screenshot.jpg**



Step 12: Establish the relationships between the models in Sequelize.

Right now, although we have established the foreign keys in the database, sequelize doesn't know about them. We need a way to tell Sequelize that pets have owners in the `web_user` table.

We do this by creating relationships using the belongsTo(…) method.

Add the following include to the top of the `models/pet.js` file:

```
const userModel = include('models/web_user');
```

Add the following 2 lines to just before the module.exports line on the bottom of the `models/pet.js` file:

```
petModel.belongsTo(userModel , { as: 'owner', timestamps: false, foreignKey:
'web_user_id'});
userModel.hasMany(petModel , { as: 'pets', timestamps: false, foreignKey:
'web_user_id'});
```

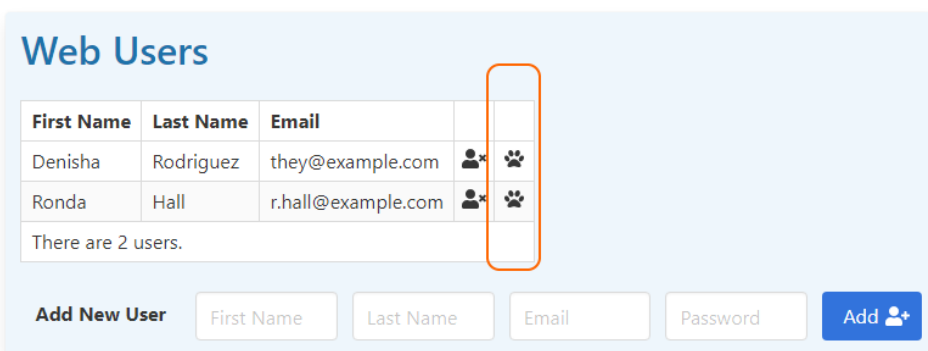
With this, now Sequelize will know the relationship between the pet and web_user tables.
These 2 lines above indicate that pets `belongTo` and have a foreign key of `web_user_id` that links to the `web_user` table. Also, each user can have multiple pets (again using the `web_user_id` foreign key).

Sequelize will create a getter for the `userModel` called `getPets()` (it's called getPets because we used `as: 'pets'` in the options) which will return a list of all the pets associated with that user.

Also, it will go the other way too. If we have a pet and we want to know its owner, we can call `petModel.getOwner()` to get back the web_user that it belongs to.

Step 13: Show all the pets that belong to a specific user.

We are going to add a link from the Web User page to the all the pets for that user.



Add a new column to the output of the Web User page (`views/index.ejs`).

Add `<th>   </th>` to the `<thead>` section as column header.

```
<thead>
<tr>
    <th> Firstname </th>
    <th> Lastname </th>
    <th> Email </th>
    <th>   </th>
    <th>   </th>
</tr>
</thead>
```

Add a link to the `/showPets` page that will pass in the id of the current `web_user`.

```
<td><a href="/showPets?id=<%= allUsers[i].web_user_id %>">
        <span class="icon is-small">
                <i class="fas fa-paw"></i>
        </span></a>
</td>
```

```
    <td><a href="/deleteUser?id=<%= allUsers[i].web_user_id %>">
        <span class="icon is-small">
            <i class="fas fa-user-times"></i>
        </span></a>
    </td>
    <td><a href="/showPets?id=<%= allUsers[i].web_user_id %>">
        <span class="icon is-small">
            <i class="fas fa-paw"></i>
        </span></a>
    </td>
</tr>
```

And let's change our `colspan` to `5` for our `<tfoot>` section.

```
<tfoot>
    <tr> <td colspan="5">
    There are <%= i %> users.
    </td> </tr>
</tfoot>
```

Step 14: Create a new route for the showPets page.

Modify `routes/router.js` to create a route to receive the `GET` request on `/showPets`:

```
router.get('/showPets', async (req, res) => {
	console.log("page hit");
	try {
		let userId = req.query.id;
		const user = await userModel.findByPk(userId);
		if (user === null) {
			res.render('error', {message: 'Error connecting to MySQL'});
			console.log("Error connecting to userModel");
		}
		else {
			let pets = await user.getPets();
			console.log(pets);
			let owner = await pets[0].getOwner();
			console.log(owner);

			res.render('pets', {allPets: pets});
		}
	}
	catch(ex) {
		res.render('error', {message: 'Error connecting to MySQL'});
		console.log("Error connecting to MySQL");
		console.log(ex);
	}
});
```

This will get the `web_user` based on the user id passed in using the `userModel.findByPk(…)` function.
For example: `/showPets?`**`id=4`**.

Next it uses that user object (assuming that user exists in the database) to get all that user's pets using the `getPets()` method.

Submit a text file with the URL of your Hosted App.
Ex: `https://fancy-style-quokka.onrender.com/`

Submit

Filename: **02_Sequelize_Web_Users_Site.txt**

Submit a zip file with your code for your Node.js App.

Submit

Filename: **03_Sequelize_Web_Users_Code.zip**
(excluding the `node_modules/` folder)

The contents of the zip file should contain the following files:

- models
- public
- routes
- views
- .gitignore
- databaseAccessLayer.js
- databaseConnection.js
- databaseConnectionSequelize.js
- index.js
- package.json
- package-lock.json
- Procfile
- staticData.js

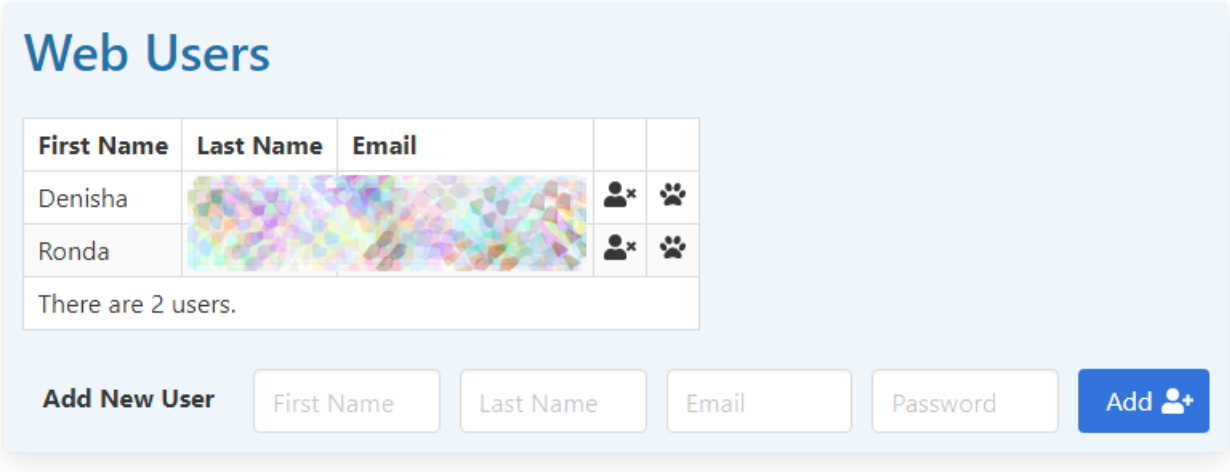
IMPORTANT!! Make sure you **exclude** the `node_modules/` folder!

The `node_modules/` folder can get very large (over 70M) and can easily be re-downloaded using `npm install`.

Submit a screenshot of your Hosted App with some users added to the database.

Submit

Filename: **04_Sequelize_Web_Users_Screenshot.jpg**



Whoohoo! All Done!